

INTERNET-OF-THINGS AND EMBEDDED SYSTEM PRACTICE

Module Code: CSC83309

LECTURE

TUYISHIMIRE Joseph

tuyishimirejo1992@gmail.com

Chapter 4: Programming for IoT Devices



4.1 Introduction to Arduino Programming

What is Arduino?

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It uses microcontrollers to read inputs (e.g., light on a sensor, a finger on a button) and turn them into outputs (e.g., activating a motor, turning on an LED).

Arduino Board Components:

Microcontroller (e.g., ATmega328)

Digital and analog input/output pins

Power supply pins

USB interface for programming and communication



Arduino IDE:

Integrated Development Environment to write, compile, and upload code to the Arduino board.

Supports C/C++ based language with built-in libraries for easy programming.

Basic Arduino Program Structure:

```
void setup() {  
  // Initialization code runs once  
}
```

```
void loop() {  
  // Main code runs repeatedly  
}
```



Example: Blinking an LED

```
void setup() {  
  pinMode(13, OUTPUT); // Set pin 13 as output  
}
```

```
void loop() {  
  digitalWrite(13, HIGH); // Turn LED on  
  delay(1000);           // Wait for 1 second  
  digitalWrite(13, LOW); // Turn LED off  
  delay(1000);           // Wait for 1 second  
}
```



4.2 Basics of Python Programming for IoT

Why Python for IoT?

Python is popular in IoT for its simplicity, readability, and the availability of powerful libraries for device control, data processing, and networking.

Python on IoT Devices:

MicroPython and CircuitPython are lightweight versions of Python designed to run on microcontrollers.

Python can also run on single-board computers like Raspberry Pi.



Key Python Concepts for IoT:

- Variables, data types, and control structures (if, loops)
- Functions and modules
- Reading sensor data using libraries (e.g., GPIO, smbus)
- Networking libraries (e.g., socket, MQTT clients)

Example: Reading a sensor value (pseudo-code):

```
import machine  
import time
```

```
sensor = machine.ADC(0) # Analog sensor on ADC pin 0
```

```
while True:  
    value = sensor.read()  
    print("Sensor Value:", value)  
    time.sleep(1)
```

4.3 Interfacing Sensors and Actuators with Microcontrollers

- **Sensors:** Devices that detect physical changes (temperature, light, humidity, motion) and convert them into electrical signals.
- **Actuators:** Devices that perform actions based on signals (motors, LEDs, buzzers).
- **Types of Sensors:**
 - Analog sensors: Output continuous voltage (e.g., temperature sensors like LM35)
 - Digital sensors: Output discrete signals (e.g., PIR motion sensors)

Connecting Sensors and Actuators:

- Use appropriate pins (analog or digital) on the microcontroller.
- Power the sensor/actuator according to specifications.
- Use pull-up or pull-down resistors if required.

Sample Arduino Code to Read a Temperature Sensor and Control an LED:

```
int sensorPin = A0; // Analog pin connected to sensor
int ledPin = 13;    // LED pin
```

```
void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  int sensorValue = analogRead(sensorPin);
  float voltage = sensorValue * (5.0 / 1023.0);
  float temperatureC = voltage * 100; // Example conversion for LM35
```

Arduino Code to Read a Temperature Sensor and Control an LED:

CONT.....

```
Serial.println(temperatureC);  
if(temperatureC > 30) {  
    digitalWrite(ledPin, HIGH); // Turn LED on if temp >  
30°C  
} else {  
    digitalWrite(ledPin, LOW); // Else turn it off  
}  
delay(1000);  
}
```

4.4 Developing Simple IoT Applications

IoT Application Components:

- ❑ Data acquisition from sensors
- ❑ Data processing and decision making
- ❑ Actuator control (if applicable)
- ❑ Data communication to cloud or local server
- ❑ Data visualization and user interface

Example Project: Smart Temperature Monitoring System

- ❑ Use Arduino or Raspberry Pi to read temperature sensor data.
- ❑ Send data to a cloud platform (e.g., ThingSpeak, AWS IoT) via Wi-Fi or Ethernet.
- ❑ Use Python scripts or Arduino code to manage data sending.
- ❑ Visualize data on a web dashboard.
- ❑ Implement simple alerts using LEDs or notifications when temperature crosses threshold

Steps to Develop:

- Setup hardware and sensor connections.
- Write code to read sensor data.
- Establish network connectivity (Wi-Fi/Ethernet).
- Send data to cloud or local server using protocols like HTTP, MQTT.
- Implement user interface or visualization tool.
- Test and calibrate system